



EVALUACIÓN FINAL - DESAFÍO DEL CURSO

Fundamentos de Programación - CIIN1205P

I. DATOS INFORMATIVOS

Título del desafío	Del problema al programa: Automatiza tu mundo con Fundamentos de Programación				
Curso	Fundamentos de Programación	Código del curso	CIIN1205P	Relevancia	Intermedia
Tipo de actividad	Informe técnico + Código fuente + Sustentación oral			Ponderación	60%

Competencia	Criterio
Resolución de problemas de ingeniería	Análisis del problema. Generación y selección de soluciones. Implementación y evaluación de resultados
Aprendizaje permanente	Reconocimiento de la necesidad de aprender. Uso de estrategias básicas de aprendizaje. Aplicación del conocimiento nuevo

II.A DESCRIPCIÓN DEL DESAFÍO

El contexto

Según el INEI (2023), el 78% de las microempresas peruanas aún gestionan sus procesos de forma manual, generando pérdidas de tiempo y errores frecuentes. En Lima Metropolitana, negocios como bodegas, librerías y talleres mecánicos procesan en promedio 50-100 transacciones diarias sin ningún sistema de registro digital. El Ministerio de la Producción reporta que la automatización básica puede reducir errores operativos en un 40% y mejorar la productividad en un 25%. En el distrito de Los Olivos, doña Carmen Quispe administra la bodega 'El Ahorro' desde hace 15 años, registrando manualmente ventas, fiados y stock en cuadernos que frecuentemente se deterioran o extravían.

En tu comunidad, pueden existir contextos similares que requieren de tu apoyo con una solución digital que automatice diversas actividades.

El reto consiste en identificar un proceso similar en tu entorno inmediato (bodega, librería, taller, academia, etc.), analizarlo sistemáticamente, diseñar la solución mediante diagramas de flujo y pseudocódigo, e implementar un prototipo funcional en Python o C# que demuestre la viabilidad de la automatización.

TU MISIÓN

Analizar un proceso manual de tu entorno y desarrollar una solución en Python o C# que incluya análisis IPO, diagrama de flujo, pseudocódigo y programa funcional que permita automatizar sus operaciones principales mediante arreglos, estructuras, manejo de datos, funciones y archivos.

Recursos de apoyo

Joyanes Aguilar, Luis (2020). Fundamentos de programación: Algoritmos, estructura de datos y objetos. McGraw-Hill. <https://ebooks724.bibliotecaupn.elogim.com:443/?il=10409>

Python Software Foundation (2024). *Tutorial oficial de Python 3*. Python.org. <https://docs.python.org/es/3/tutorial/index.html>

II.B PRÁCTICA DE CAMPO

Entrenamiento autónomo durante el desafío. Completa estos pasos con tu equipo. Cada salida se integra directamente en tu entregable final.

#	Paso	Qué hacer	Salida	Destino
1	Identificar el proceso a automatizar	Visita un negocio o espacio de tu entorno (bodega, librería, taller, academia, hogar, etc.). Observa durante 30 minutos cómo se realiza un proceso manual repetitivo. Elabora el diagrama de Ishikawa para identificar el problema. Entrevista brevemente al encargado preguntando: ¿Cuántas veces al día hace esta tarea? ¿Qué problemas tiene? ¿Cuánto tiempo le toma? Documenta con fotos o notas el proceso actual.	Ficha de observación con: nombre del proceso, lugar, persona entrevistada, frecuencia, problemas detectados y 3-5 fotos del proceso actual. Diagrama de Ishikawa	Informe [1.1] Descripción del problema
2	Analizar entradas, procesos y salidas	Descompón el proceso observado identificando: (1) ENTRADAS: qué datos se reciben (nombre, cantidad, precio, fecha, etc.) (Más de 5 entradas, más de 5 procesos y más de 3 salidas), (2) PROCESOS: qué operaciones se realizan (calcular, comparar, buscar, ordenar, etc.), (3) SALIDAS: qué resultados se generan (totales, reportes, alertas, etc.). Elabora una tabla que muestre cada componente con ejemplos concretos del proceso real.	Tabla de análisis IPO (Entrada-Proceso-Salida) con mínimo 5 operaciones identificadas y ejemplos de datos reales	Informe [1.2] Análisis del problema
2.1	Matriz de trazabilidad funcional (obligatoria)	Construye una matriz que vincule requerimientos funcionales con entradas, procesos (IPO), funciones implementadas, opciones del menú y el caso de prueba que valida cada requerimiento. ABET: La ausencia de trazabilidad explícita limita el nivel máximo a INTERMEDIO.	Matriz de trazabilidad completa (tabla) Formato: Req. funcional Entrada Proceso Función Opción menú Prueba.	Informe [1.2] Análisis del problema Incluir como subsección [1.2.1] Matriz de trazabilidad (o dentro de [1.2]).
3	Diseñar el diagrama de flujo	Usando PSeInt o draw.io, construye el diagrama de flujo de la solución. Incluye: símbolo de inicio/fin, entradas de datos, procesos de cálculo, estructuras condicionales (SI-ENTONCES-SINO) para decisiones, estructuras repetitivas (MIENTRAS o PARA) para operaciones múltiples, y salidas de información. El diagrama debe reflejar la lógica completa del proceso identificado.	Diagrama de flujo exportado en imagen (PNG o JPG) con todos los símbolos correctamente conectados	Informe [1.3] Diagrama de flujo
4	Elaborar el pseudocódigo	Traduce el diagrama de flujo a pseudocódigo estructurado.	Documento con pseudocódigo completo	Informe [1.4] Pseudocódigo

#	Paso	Qué hacer	Salida	Destino
		Utiliza las convenciones estándar: ALGORITMO nombre, VARIABLES, INICIO, LEER, ESCRIBIR, SI-ENTONCES-SINO, MIENTRAS-HACER, PARA-HASTA, FIN. El pseudocódigo debe ser claro, indentado correctamente y debe incluir comentarios explicativos en las secciones principales.	(mínimo 60 líneas) con comentarios explicativos	
5	Implementar el código en Python o C#	Traduce el pseudocódigo al lenguaje elegido (Python o C#). Implementa: variables con tipos de datos apropiados, entrada de datos por teclado, estructuras condicionales (if-else), estructuras repetitivas (while, for), funciones básicas para organizar el código, y manejo de listas/arreglos para almacenar datos. Programar solución con menú con ≥ 5 opciones, funciones, arreglos, archivos, etc. El código debe estar comentado y seguir buenas prácticas de nombrado. <i>ABET: Debe manejar al menos 50 registros simulados (p. ej., 50 ventas/items) sin fallos ni pérdida de datos.</i> <i>ABET: Implementa validación de entradas (tipo, rango, no vacío) con mensaje y reintento sin cerrar el programa.</i>	Archivo de código fuente (.py o .cs) con mínimo 100 líneas funcionales y comentarios. Evidencia adicional: captura/registro de ejecución con 50 registros (simulados o ingresados).	ZIP/src/ + Informe [1.5] Código implementado
6	Probar y validar el programa	Ejecuta el programa con al menos 10 pruebas (6 normales + 2 límite + 2 inválidas), casos de prueba diferentes: datos normales, valores límite (0, negativos, muy grandes), datos incorrectos. Documenta cada prueba con: datos de entrada, resultado esperado, resultado obtenido, estado (PASÓ/FALLÓ). Corrige los errores encontrados y vuelve a probar. <i>ABET: Justifica la selección de los casos límite e inválidos (por qué esos valores).</i> <i>ABET: Identifica al menos 1 error detectado y documenta la corrección (antes/después).</i>	Tabla de pruebas con ≥ 10 casos (6 normales + 2 límite + 2 inválidas) y capturas de pantalla de la ejecución del programa. Incluye columna 'Tipo de caso' (Normal/Límite/Inválido) para auditoría.	Informe [1.6] Pruebas y resultados
7	Reflexionar sobre el impacto y ética	Responde por escrito: (1) ¿Cómo beneficia esta solución al usuario del proceso original? (2) ¿Qué datos personales maneja el programa y cómo se protegen? (3) ¿Qué limitaciones tiene tu solución actual? (4) ¿Qué mejoras implementarías con más tiempo? (5) ¿Qué aprendió cada integrante del equipo durante el proyecto? Adjuntar evidencias de autoaprendizaje y mejora: (a) bitácora breve, (b) links usados, (c) "antes/después" (capturas o fragmentos) <i>ABET: Incluye 1 limitación técnica concreta del algoritmo y 1 mejora estructural futura (con justificación técnica).</i> <i>ABET: Describe 1 riesgo asociado a los datos (privacidad/seguridad) y cómo lo mitigaste en el prototipo.</i>	Documento de reflexión (1 página) con respuestas fundamentadas a las 5 preguntas	Informe [1.7] Reflexión técnica y ética Informe [1.8] Evidencias de autoaprendizaje y mejora

Importante: Lo que produces en la práctica de campo ES parte del informe. No reescribas desde cero: edita y mejora lo que ya tienes. Las fotos, tablas, diagramas y código que generas en cada paso van directamente a las secciones correspondientes del informe final.

II.C MOMENTOS SINCRÓNICOS

Durante las sesiones presenciales trabajarás con tu docente en estas actividades guiadas:

#	Actividad
1	Identificar el problema y los procesos automatizables del entorno mediante la técnica de lluvia de ideas grupal y diagrama de Ishikawa, seleccionando el más viable según criterios de frecuencia, complejidad y acceso a información.
2	Analizar el proceso seleccionado descomponiéndolo en entradas, operaciones y salidas, utilizando la técnica IPO (Input-Process-Output) con retroalimentación del docente. Y genera y selecciona soluciones.
3	Representar la solución mediante diagramas de flujo en PSeInt, aplicando correctamente los símbolos de inicio/fin, decisión, proceso y flujo de datos.
4	Implementar el pseudocódigo en Python o C# durante sesión guiada de laboratorio, resolviendo dudas de sintaxis y estructuras de control en tiempo real.
5	Comparar diferentes enfoques de solución entre equipos, analizando ventajas y desventajas de cada implementación en términos de claridad y eficiencia.
6	Integrar la retroalimentación recibida en revisiones parciales para mejorar la documentación, el código y la estructura general del proyecto.
7	Comunicar los resultados del proyecto mediante sustentación oral, explicando la lógica técnica empleada y demostrando el funcionamiento del programa.

II.D LABORATORIO

Las horas de laboratorio son espacios guiados donde practicarás con PSeInt, Python y C# bajo supervisión del docente. Aprenderás a: configurar el entorno de desarrollo, crear diagramas de flujo, escribir pseudocódigo estructurado, implementar estructuras de control, depurar errores comunes y documentar tu código. Cada sesión de laboratorio alimenta directamente tu proyecto final. Revisa las guías de laboratorio propuestas para el curso.

III. RÚBRICA DE EVALUACIÓN

Criterio	No logrado	INICIAL	INTERMEDIO	AVANZADO
Análisis del problema	0	0.5	1	2
	No identifica claramente el problema ni reconoce sus características esenciales.	Identifica la existencia del problema (a través del diagrama de Ishikawa y descripción), mencionando sus características, con causas y alcance incompleto.	Delimita el problema con Ishikawa completo (causas principales) y describe consecuencias; incluye evidencia mínima del proceso (entrevista/notas/fotos) y parámetros críticos a controlar.	Explica el problema con enfoque de ingeniería: Ishikawa con causas raíz, alcance y supuestos; cuantifica impacto (frecuencia/tiempo/errores) con datos recolectados y justifica por qué la automatización es necesaria.
Generación y selección de soluciones	0	0.5	1	2
	No propone alternativas de solución o las menciona sin relación con el problema ni sus requerimientos.	Propone 1 alternativa e indica datos y operaciones principales, con beneficios generales.	Propone ≥ 2 alternativas y selecciona 1 comparando ≥ 2 criterios (p. ej., simplicidad, tiempo, riesgo, datos). La selección está justificada y coherente con el análisis IPO.	Propone ≥ 2 alternativas y selecciona 1 usando ≥ 2 criterios técnicos medibles (claridad/mantenibilidad, tiempo de ejecución, uso de datos/errores, escalabilidad básica). Adjunta Matriz de Trazabilidad (Req $\rightarrow$ Menú/Función $\rightarrow$ Prueba) completa y consistente.
Implementación y evaluación de resultados	0	0.5	1	2
	No entrega software ejecutable o no demuestra ejecución funcional.	Ejecuta el programa y demuestra ≥ 2 opciones del menú, pero sin tabla de pruebas ni evidencias de validación.	Presenta programa ejecutable con menú (≥ 3 opciones) + ≥ 2 funciones + uso de arreglo/lista y manejo de cadenas; adjunta tabla de pruebas con ≥ 6 casos (entrada–esperado–obtenido).	Presenta programa ejecutable con menú (≥ 5 opciones) y ≥ 5 funciones cohesionadas (sin duplicación). Usa arreglo/lista, cadenas y persistencia en archivo (leer/escribir). Maneja ≥ 50 registros simulados sin fallos. Incluye validación robusta (tipo/rango/no vacío) con mensaje y reintento. Adjunta ≥ 10 pruebas (6 normales + 2 límite + 2 inválidas) documentadas.
Uso de estrategias básicas de aprendizaje	0	0.5	1	1.5
	No presenta evidencias de	Presenta 1 evidencia (apunte/enlace/capt)	Presenta ≥ 2 evidencias y describe	Presenta ≥ 3 evidencias + bitácora breve (fecha–

Criterio	No logrado	INICIAL	INTERMEDIO	AVANZADO
	autoaprendizaje (sin registro, sin fuentes, sin aplicación).	ura) y describe 1 aprendizaje aplicado (qué cambió en el código).	≥2 aprendizajes aplicados, indicando dónde se aplicaron (sección/función del código).	brecha–acción–resultado) y links usados; muestra evidencia “antes/después” (capturas/fragmentos) de mejoras aplicadas.
Reconocimiento de la necesidad de aprender	0	0.5	1	1.5
	No identifica brechas (no señala qué no sabía o qué tuvo que aprender).	Identifica 1 brecha relevante (p. ej., archivos, arreglos, funciones) y explica por qué era necesaria para el proyecto.	Identifica ≥2 brechas y define acciones de aprendizaje (tutorial, práctica, documentación) con evidencia de ejecución.	Identifica ≥3 brechas, las prioriza (críticas vs. secundarias) y documenta resultados con evidencia “antes/después” de la solución aplicada.
Aplicación del conocimiento nuevo	0	0.5	1	1.5
	No incorpora conocimiento nuevo; se limita a una solución mínima sin mejoras verificables.	Incorpora 1 elemento nuevo respecto al primer avance (p. ej., función, validación, arreglo) y evidencia su uso.	Incorpora ≥2 elementos nuevos y demuestra su funcionamiento con ejemplos de ejecución (capturas/log) e indica en qué parte del código se aplican.	Incorpora ≥3 elementos nuevos (p. ej., archivo + validación + modularidad + búsqueda/ordenamiento o opcional) y explica su propósito e impacto (mejor control, menor error, mayor claridad) en el informe.
Claridad del diseño del algoritmo	0	1.5	3	4.5
	No entrega diagrama de flujo ni pseudocódigo.	Entrega diagrama de flujo o pseudocódigo (solo uno) y cubre ≥1 opción del menú.	Entrega diagrama de flujo + pseudocódigo; ambos cubren todas las opciones del menú (≥3). El pseudocódigo tiene ≥50 líneas e incluye secuencia, condicional y repetición.	Entrega diagrama de flujo y pseudocódigo que cubren todas las opciones del menú (≥5). El pseudocódigo tiene ≥60 líneas. Existe correspondencia directa diagrama ↔ pseudocódigo ↔ funciones (1:1 entre opción de menú y función). El diagrama usa simbología estándar y no hay contradicciones con el código.
Corrección y eficiencia básica del algoritmo	0	1	2.5	5
	El algoritmo no resuelve el problema o tiene errores que impiden su	Ejecuta el caso principal y produce salida para ≥3 casos normales, sin evidencia de límite/inválidos.	Ejecuta ≥6 casos normales y ≥2 casos límite o inválidos; registra esperado/obtenido e incluye validaciones	Ejecuta y documenta 10 pruebas (6 normales + 2 límite + 2 inválidas) con coincidencia esperado/obtenido. Implementa validación

Criterio	No logrado	INICIAL	INTERMEDIO	AVANZADO
	funcionamiento.		básicas (no vacío, rango).	robusta y manejo de error (mensaje + reintento) sin terminar el programa. Evidencia corrección iterativa (al menos 1 fallo y su corrección “antes/después”) y evita redundancias usando funciones.

IV. CONSIDERACIONES PARA LA ELABORACIÓN DEL ENTREGABLE

1. Informe de proyecto computacional (PDF)

Estructura tu informe usando las salidas de cada paso de la práctica de campo:

Parte del entregable	Qué incluir	Evidencia (Práctica de campo)
[1.1] Descripción del problema	Contexto del proceso observado, lugar, persona entrevistada, frecuencia del proceso, problemas detectados, fotos del proceso actual y justificación de por qué es relevante automatizarlo. Diagrama de Ishikawa.	Paso 1
[1.2] Análisis del problema	Generación y selección de soluciones. Tabla IPO completa con entradas, procesos y salidas identificadas. Ejemplos concretos de datos reales del proceso	Paso 2
[1.3] Diagrama de flujo	Imagen del diagrama de flujo con todos los símbolos correctamente utilizados: inicio/fin, entrada/salida, proceso, decisión, bucles	Paso 3
[1.4] Pseudocódigo	Pseudocódigo estructurado completo con comentarios explicativos, correctamente indentado	Paso 4
[1.5] Código implementado	Capturas del código fuente en Python o C# con comentarios visibles. Explicación breve de las estructuras utilizadas	Paso 5
[1.6] Pruebas y resultados	Tabla de casos de prueba y capturas de pantalla de la ejecución mostrando diferentes escenarios	Paso 6
[1.7] Reflexión técnica y ética	Respuestas fundamentadas a las 5 preguntas de reflexión sobre impacto, ética, limitaciones, mejoras y aprendizajes del equipo	Paso 7
[1.8] Evidencias de autoaprendizaje y mejora	Bitácora breve (fecha–brecha–acción–resultado), links usados, y evidencia 'antes/después' (capturas/fragmentos/commits) mostrando aprendizajes aplicados. <i>ABET</i> : Esta sección sustenta Aprendizaje permanente (brechas, estrategias y aplicación).	Paso 7

2. Código fuente (incluir en PDF o adjunto)

Organiza tu código fuente de la siguiente manera:

Parte del entregable	Qué incluir	Evidencia (Práctica de campo)
Archivo principal	Código fuente completo (.py o .cs) con comentarios en cada sección principal	Paso 5
Documentación	README breve explicando cómo ejecutar el programa y qué dependencias necesita	Paso 5

3. Presentación para sustentación (PPT o PDF)

Organiza tu presentación basándote en los pasos de la práctica:

Parte del entregable	Qué incluir	Evidencia (Práctica de campo)
Slide 1: Portada	Título del proyecto, integrantes, curso y fecha	N/A
Slides 2-3: El problema	Descripción del proceso observado con fotos y datos de la entrevista	Paso 1
Slides 4-5: Análisis	Tabla IPO resumida y puntos clave del análisis	Paso 2
Slides 6-7: Diseño	Diagrama de flujo y extracto del pseudocódigo más relevante	Paso 3 y 4
Slides 8-9: Implementación	Capturas del código con las estructuras principales resaltadas	Paso 5
Slide 10: Demo en vivo	Indicación de que se hará demostración del programa funcionando	Paso 6
Slides 11-12: Conclusiones	Reflexiones, aprendizajes del equipo y mejoras futuras	Paso 7

Requisitos:

- Máximo 12-15 slides
- Usar imágenes y capturas, evitar texto excesivo
- Preparar demo en vivo del programa

4. Sustentación oral

Prepara tu exposición cubriendo cada fase del trabajo:

Parte del entregable	Qué incluir	Evidencia (Práctica de campo)
Introducción (2-3 min)	Presentación del equipo, contexto del problema + Ishikawa y proceso observado	Paso 1
Análisis y diseño (4-5 min)	Explicación del análisis IPO, diagrama de flujo y pseudocódigo	Paso 2, 3 y 4
Demostración (4-5 min)	Ejecución en vivo del programa con diferentes casos de prueba	Paso 5 y 6
Conclusiones (2-3 min)	Reflexiones, lecciones aprendidas y propuestas de mejora	Paso 7

Requisitos:

- Duración total: 12-15 minutos
- Todos los integrantes deben participar activamente
- Preparar el entorno para la demo antes de iniciar
- Estar preparados para responder preguntas del docente

V. CONDICIONES DE ENTREGA

Formato	PDF + PPT o PDF
Extensión	No especificado
Modalidad	En equipos
Semana	Evaluación Final
Nombre archivo	Nombre del grupo + código del curso_EF
Medio de entrega	Aula virtual institucional